4.5. Python и управление системами защиты

4. Python в задачах информационной безопасности

В этом уроке

- Узнаем зачем использовать Python в ИБ
- Разберём сценарии мониторинга и аудита
- Рассмотрим примеры автоматизации
- Изучим сценарии ограничения/отключения доступа

Роль Python в ИБ-среде

- Почему Python удобен в ИБ?
- Кроссплатформенность: Python работает одинаково хорошо на Windows, Linux и macOS. Это удобно при администрировании и автоматизации в разнородных средах.
- Простота синтаксиса: Код легко читается и быстро пишется — это критично при создании оперативных решений для аудита и мониторинга.
- Большое количество библиотек:
  - psutil — доступ к информации о системе (CPU, память, процессы).
  - os и subprocess — выполнение системных команд.
  - paramiko — SSH-доступ к удалённым машинам.
  - logging, email, smtplib — логирование и уведомления
- Интеграция с SIEM и логами: Python-скрипты легко встраиваются в существующие системы мониторинга и анализа событий безопасности

Сценарии управления ресурсами

- Типовые сценарии:
- Контроль загрузки:
  - CPU, память, диск.
  - Реакция при превышении порогов (например, отправить уведомление).
- Мониторинг процессов:
  - Список активных процессов.
  - Выявление подозрительных/нежелательных (например, netcat, nmap).
- Управление пользователями и службами:
  - Временное отключение доступа.
  - Проверка, кто в системе.
  - Уведомления и логирование:
  - Отправка email / логов при отклонениях.

```
import psutil


print(f"CPU: {psutil.cpu_percent()}%")
print(f"Доступная память: {psutil.virtual_memory().available / 1024**2:.2f} MB")
```

Что и как проверять?

- Аудит — ключевой элемент в обнаружении попыток вторжения или несанкционированных действий.
- Основные цели аудита:
- Контроль доступа:
  - Кто входит в систему?
  - Были ли неудачные попытки входа?
- Анализ логов:
  - Проверка системных журналов (например, /var/log/auth.log, Windows Security.log)
  - Поиск шаблонов (неудачные входы, входы под root, SSH-доступ).
- Инструменты Python:
  - os — взаимодействие с файловой системой.
  - subprocess — вызов команд (например, last, who, netstat).
  - logging — организация аудита внутри скрипта.

Аудит активности пользователей

Python-скрипт для поиска попыток входа

```
with open("/var/log/auth.log", "r") as f:
    for line in f:
        if "Failed password" in line or "Accepted password" in line:
            print(line)
```

Что происходит?

- Чтение логов построчно.
- Фильтрация событий по ключевым словам (Failed, Accepted).
- Можно модифицировать фильтр: по IP, имени пользователя, времени.

Автоматизация действий

Python позволяет оперативно реагировать на события без участия администратора.

Типовые автоматизированные действия:

🔒 Блокировка IP-адресов после нескольких неудачных входов (например, через iptables, Windows Firewall).

⛔ Завершение подозрительных процессов (например, ncat, python3 reverse_shell.py).

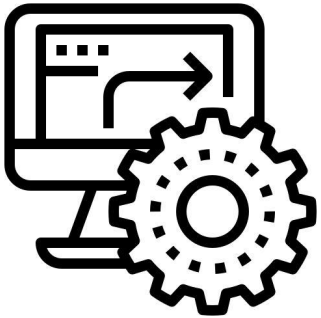
🛑 Отключение пользователя при подозрительной активности.

📧 Оповещение администратора по email или в Telegram.

Автоматизация действий

Также возможна интеграция Вашего скрипта с планировщиком ОС через cron (Linux) или Task Scheduler (Windows) для запуска скриптов по расписанию или при триггере

```
import subprocess

subprocess.run(["sudo", "ufw", "deny", "from", "192.168.1.42"])
```



Мониторинг системных параметров

Отслеживаем загрузку CPU и памяти

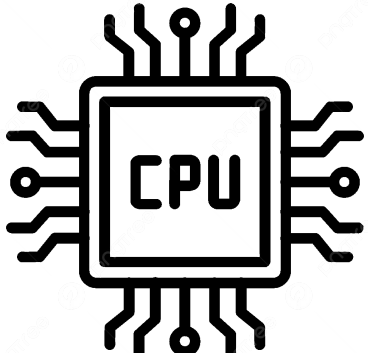
```
import psutil
import smtplib

if psutil.cpu_percent() > 85:
    with smtplib.SMTP("smtp.example.com") as server:
        server.sendmail("admin@example.com", "Subject: CPU warning\n\nHigh CPU load!")
```

Что делает:

- Проверяет загрузку CPU.
- Если выше 85%, отправляет email.

Возможности расширения:

- Проверка дискового пространства.
- Поддержка телеграм-уведомлений (через API).
- Логирование в файл или базу данных.



Отключение доступа пользователю

В Linux можно временно отключить пользователя (например, usermod -L login).

В Windows — отключить учётную запись через net user.

```
import subprocess


user = "testuser"
subprocess.run(["sudo", "usermod", "-L", user])
```

Пояснение:

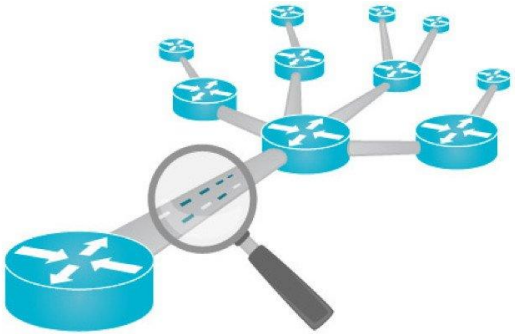
·Блокирует пользователя testuser (учётная запись остаётся, но вход невозможен).

·Может быть вызван автоматически при подозрительных действиях (много неудачных входов, действия вне расписания и т.п.).

Анализ сетевой активности

Цель: Понять, кто устанавливает соединения, какие порты слушаются, откуда приходят запросы.

```
import psutil

for conn in psutil.net_connections(kind='inet'):
    print(f"{conn.laddr} → {conn.raddr} (status: {conn.status})")
```



Что показывает:
- Локальные и удалённые адреса.
- Статус соединения (LISTEN, ESTABLISHED).
- Можно отслеживать необычные подключения (например, к нехарактерным IP или портам).

Применение:
- Поиск троянов/бэкдоров, устанавливающих соединения наружу.
- Интеграция с системой реагирования (закрытие порта, оповещение).

Хранение и обработка результатов

Варианты хранения:

Файлы CSV / JSON: просто, удобно для импорта в Excel, pandas и пр.

SQLite: встроенная СУБД без отдельного сервера, удобно для хранения логов.

Удалённые базы (MySQL/PostgreSQL): если нужно централизованное хранилище.

```
import sqlite3
conn = sqlite3.connect("audit.db")
c = conn.cursor()
c.execute("CREATE TABLE IF NOT EXISTS log (event TEXT, ts TIMESTAMP)")
c.execute("INSERT INTO log VALUES (?, datetime('now'))", ("User login failed",))
conn.commit()
```

Реагирование на события

Python-скрипты могут запускаться не только по расписанию, но и по конкретным событиям в системе:

- 💻 Попытка входа в систему (/var/log/auth.log или Event Viewer в Windows).
- 📁 Создание файла в подозрительной директории (например, в Downloads).
- ⚠️ Обращение к важному ресурсу или конфигурации.
- 🌐 Сетевое подключение на нестандартный порт.

Реагирование на события

Как реализовать:

- В Linux — утилиты auditd, inotify, systemd с unit-сервисами.
- В Windows — события из журнала (например, с помощью pywin32 или WMI).

```python
import time
import os

path = "/home/user/Downloads"
seen = set(os.listdir(path))

while True:
    current = set(os.listdir(path))
    new = current - seen
    if new:
        print("New file(s):", new)
    seen = current
    time.sleep(10)
```

Интеграция с системами оповещений

Важно не только зафиксировать событие, но и своевременно оповестить ответственного специалиста по ИБ через:

- 📧 Email (через smtplib).
- 💬 Telegram-бот (через Telegram Bot API).
- 📲 Slack, Discord, Mattermost (через Webhooks).
- 🔔 Запись в SIEM-систему (Security Information and Event Management).

```
import requests

token = "YOUR_BOT_TOKEN"
chat_id = "YOUR_CHAT_ID"
text = "⚠️ Подозрительная активность от 192.168.0.55"

requests.get(f"https://api.telegram.org/bot{token}/sendMessage?chat_id={chat_id}&text={text}")
```

Обнаружение запуска вредоносного ПО

Подозрительные признаки:

- Использование интерпретаторов (python, powershell, cmd) с передачей скриптов.
- Обращение к IP-адресам за пределами страны.
- Использование ключевых слов: reverse shell, exec, eval, base64.

Меры реагирования:

- Завершить процесс.
- Сохранить дамп командной строки.
- Отправить оповещение.

```
import psutil

for proc in psutil.process_iter(['pid', 'name', 'cmdline']):
    try:
        if 'nc' in proc.info['cmdline'] or 'bash -i' in proc.info['cmdline']:
            print(f"🚨 Подозрительный процесс: {proc.info}")
    except Exception:
        continue
```

Интеграция с файрволом

С помощью Python можно взаимодействовать с системами безопасности:

Примеры:

- 🔒 Блокировка IP через iptables, ufw, netsh.
- 📃 Создание временных правил доступа.
- 🔄 Удалённая смена настроек (например, через SSH + скрипты).

```
import subprocess


ip = "203.0.113.55"
subprocess.run(["sudo", "iptables", "-A", "INPUT", "-s", ip, "-j", "DROP"])
```

Итоги урока

- Узнали зачем использовать Python в ИБ
- Разобрали сценарии мониторинга и аудита
- Рассмотрели примеры автоматизации
- Изучили сценарии ограничения/отключения доступа